In-class exercise: code smells — solutions

In-class exercise — problems and solutions

Each exercise: **problem** slide → **discussion** (smells named) → **solution**. ***

Exercise 1

```
def f(x, s, n, k, a, b, flag):
    from math import sqrt, pi, exp, comb
    if flag == "normal":
        return (1/(sqrt(2*pi)*s)) * exp(-0.5*((x-a)/s)**2)
    elif flag == "binomial":
        return comb(n, k) * (x**k) * ((1-x)**(n-k))
    elif flag == "beta":
        from math import gamma
        return (x**(a-1)*(1-x)**(b-1))/(gamma(a)*gamma(b)/gamma(a+b))
```

Exercise 1 — discussion

Smells present:

Exercise 1 — discussion

Smells present:

- 🏷️ **Mysterious names** — `f`, `x`, `s`, `a`, `b` communicate nothing.

Exercise 1 — discussion

Smells present:

- 🏷️ **Mysterious names** — `f`, `x`, `s`, `a`, `b` communicate nothing.
- 🎛️ **Too many parameters** — seven args; several only used in one branch.

Exercise 1 — discussion

Smells present:

- 🏷️ **Mysterious names** — `f`, `x`, `s`, `a`, `b` communicate nothing.
- 🎛️ **Too many parameters** — seven args; several only used in one branch.
- 🧩 **Shotgun surgery** risk — adding a distribution means editing this function.

Exercise 1 — discussion

Smells present:

- 🏷️ **Mysterious names** — `f`, `x`, `s`, `a`, `b` communicate nothing.
- 🎛️ **Too many parameters** — seven args; several only used in one branch.
- 🧩 **Shotgun surgery** risk — adding a distribution means editing this function.

Exercise 1 — discussion

Smells present:

- 🏷️ **Mysterious names** — `f`, `x`, `s`, `a`, `b` communicate nothing.
- 🎛️ **Too many parameters** — seven args; several only used in one branch.
- 🧩 **Shotgun surgery** risk — adding a distribution means editing this function.

Refactoring direction: Separate functions per distribution, each with descriptive parameter names.

***

Exercise 1 — one possible solution

```
from math import sqrt, pi, exp, comb, gamma

def normal_pdf(x, mean, sd):
    return (1/(sqrt(2*pi)*sd)) * exp(-0.5*((x-mean)/sd)**2)

def binomial_pmf(prob, n, k):
    return comb(n, k) * (prob**k) * ((1-prob)**(n-k))

def beta_pdf(x, alpha, beta_):
    B = gamma(alpha)*gamma(beta_)/gamma(alpha+beta_)
    return (x**(alpha-1) * (1-x)**(beta_-1)) / B
```

Exercise 2

```
def run(data, conds, sbjs, dprimes, criteria, mdl, fit, r, p, out):
    r = []
    for s in sbjs:
        sd = [data[i] for i in range(len(data)) if conds[i] == s]
        dp = sum([x[0] for x in sd]) / len(sd)
        cr = sum([x[1] for x in sd]) / len(sd)
        dprimes.append(dp)
        criteria.append(cr)
    fit = mdl(dprimes, criteria)
    out = "significant" if fit.pvalue < 0.05 else "not significant"
    return out
```

Exercise 2 — discussion

Smells present:

Exercise 2 — discussion

Smells present:

- 🎛️ **Too many parameters** — ten args; several are immediately overwritten.

Exercise 2 — discussion

Smells present:

- 🎛️ **Too many parameters** — ten args; several are immediately overwritten.
- 📏 **Long method** — aggregates per-subject data, fits a model, formats output.

Exercise 2 — discussion

Smells present:

- 🎛️ **Too many parameters** — ten args; several are immediately overwritten.
- 📏 **Long method** — aggregates per-subject data, fits a model, formats output.
- 🔁 **Variable mutations** — `r`, `fit`, `out` are passed in but rebound at once.

Exercise 2 — discussion

Smells present:

- 🎛️ **Too many parameters** — ten args; several are immediately overwritten.
- 📏 **Long method** — aggregates per-subject data, fits a model, formats output.
- 🔁 **Variable mutations** — `r`, `fit`, `out` are passed in but rebound at once.

Exercise 2 — discussion

Smells present:

- 🎛️ Too many parameters — ten args; several are immediately overwritten.
- 📏 Long method — aggregates per-subject data, fits a model, formats output.
- 🔁 Variable mutations — `r`, `fit`, `out` are passed in but rebound at once.

Refactoring direction: Extract a helper for per-subject means; trim the parameter list. ***

Exercise 2 — one possible solution (part 1)

```
def subject_means(data, conditions, subject):
    rows = [data[i] for i in range(len(data)) if conditions[i] == subject]
    dprime    = sum(r[0] for r in rows) / len(rows)
    criterion = sum(r[1] for r in rows) / len(rows)
    return dprime, criterion
```

Exercise 2 — one possible solution (part 2)

```
def fit_model(data, conditions, subjects, model):
    dprimes, criteria = [], []
    for s in subjects:
        dp, cr = subject_means(data, conditions, s)
        dprimes.append(dp)
        criteria.append(cr)
    return model(dprimes, criteria)
```

Exercise 2 — one possible solution (part 2)

```
def report_result(fit):
    # ... check that fit is the right kind of object
    if fit.pvalue < 0.05
        return "significant"
    if fit.pvalue >= 0.05
        return "not significant"
    # ... this should not happen, maybe throw an error here
```

Exercise 3

```
def c(h, f):
    from scipy.stats import norm
    return norm.ppf(h) - norm.ppf(f)

def b(h, f):
    from scipy.stats import norm
    return -0.5 * (norm.ppf(h) + norm.ppf(f))
```

Exercise 3 — discussion

Smells present:

Exercise 3 — discussion

Smells present:

- 🔡 Excessively short identifiers — `c`, `b`, `h`, `f` are opaque.

Exercise 3 — discussion

Smells present:

- 🔡 **Excessively short identifiers** — `c`, `b`, `h`, `f` are opaque.
- 📋 **Duplicated code** — `norm` is imported twice inside identical boilerplate.

Exercise 3 — discussion

Smells present:

- 🔡 **Excessively short identifiers** — `c`, `b`, `h`, `f` are opaque.
- 📋 **Duplicated code** — `norm` is imported twice inside identical boilerplate.

Exercise 3 — discussion

Smells present:

- 🔡 **Excessively short identifiers** — `c`, `b`, `h`, `f` are opaque.
- 📋 **Duplicated code** — `norm` is imported twice inside identical boilerplate.

Refactoring direction: Use descriptive names; hoist the shared import. ***

Exercise 3 — one possible solution

```python
from scipy.stats import norm

def dprime(hit_rate, false_alarm_rate):
    return norm.ppf(hit_rate) - norm.ppf(false_alarm_rate)

def criterion(hit_rate, false_alarm_rate):
    return -0.5 * (norm.ppf(hit_rate) + norm.ppf(false_alarm_rate))
```

Exercise 4

```
def compute_sensitivity_index_for_signal_detection_analysis(hr, far):
    from scipy.stats import norm
    # Compute d-prime using the standard SDT formula
    # hr is hit rate, far is false alarm rate
    z_hit = norm.ppf(hr)   # z-score for hit rate
    z_far = norm.ppf(far)  # z-score for false alarm rate
    dp = z_hit - z_far     # d-prime is the difference
    return dp              # return the result
```

Exercise 4 — discussion

Smells present:

Exercise 4 — discussion

Smells present:

- 📐 **Excessively long identifier** — the name describes structure implicit from context.

Exercise 4 — discussion

Smells present:

- 📐 **Excessively long identifier** — the name describes structure implicit from context.
- 💬 **Excessive comments** — every line re-states what the code visibly does.

Exercise 4 — discussion

Smells present:

- 📐 **Excessively long identifier** — the name describes structure implicit from context.
- 💬 **Excessive comments** — every line re-states what the code visibly does.

Exercise 4 — discussion

Smells present:

- 📐 **Excessively long identifier** — the name describes structure implicit from context.
- 💬 **Excessive comments** — every line re-states what the code visibly does.

Refactoring direction: Shorter name; one docstring for the interface; no inline commentary. ***

Exercise 4 — one possible solution

```python
from scipy.stats import norm

def dprime(hit_rate, false_alarm_rate):
    """Return d-prime (SDT sensitivity index)."""
    return norm.ppf(hit_rate) - norm.ppf(false_alarm_rate)
```

Exercise 5

```python
import numpy as np

def analyze(results):
    results = [r for r in results if r is not None]
    results = [r * 1000 for r in results]
    results = np.array(results)
    results = results[results < np.percentile(results, 95)]
    results = results - results.mean()
    return results
```

Exercise 5 — discussion

Smells present:

Exercise 5 — discussion

Smells present:

- 🔁 **Variable mutations** — `results` is rebound five times; its meaning shifts on every line.

Exercise 5 — discussion

Smells present:

- 🔁 **Variable mutations** — `results` is rebound five times; its meaning shifts on every line.

Exercise 5 — discussion

Smells present:

- 🔁 Variable mutations — `results` is rebound five times; its meaning shifts on every line.

Refactoring direction: Give each transformation stage a distinct, descriptive name. ***

Exercise 5 — one possible solution

```python
import numpy as np

def preprocess_rts(raw_rts):
    valid_ms = np.array([r * 1000 for r in raw_rts if r is not None])
    trimmed  = valid_ms[valid_ms < np.percentile(valid_ms, 95)]
    centered = trimmed - trimmed.mean()
    return centered
```

Exercise 6

```
def run_experiment(participant_id, session_number,
                   condition_label, stimulus_list,
                   response_key_map, timeout_ms,
                   practice_trials, fixation_duration_ms,
                   feedback_enabled, log_file_path):
    ...
```

Exercise 6 — discussion

Smells present:

Exercise 6 — discussion

Smells present:

- 🎛️ **Too many parameters** — ten arguments; callers must track details that belong together.

Exercise 6 — discussion

Smells present:

- 🎛️ **Too many parameters** — ten arguments; callers must track details that belong together.

Exercise 6 — discussion

Smells present:

- 🎛️ **Too many parameters** — ten arguments; callers must track details that belong together.

Refactoring direction: Group related parameters into a dataclass; pass that instead. ***

Exercise 6 — one possible solution (part 1)

```
from dataclasses import dataclass

@dataclass
class ExperimentConfig:
    condition_label: str
    stimulus_list: list
    response_key_map: dict
    timeout_ms: int
    practice_trials: int
    fixation_duration_ms: int
    feedback_enabled: bool
    log_file_path: str
```

Exercise 6 — one possible solution (part 2)

```
def run_experiment(participant_id, session_number,
                   config: ExperimentConfig):
    ...
```

Exercise 7

```
hits_a  = sum(1 for t in block_a if t.correct)
total_a = len(block_a)
hr_a    = hits_a / total_a

hits_b  = sum(1 for t in block_b if t.correct)
total_b = len(block_b)
hr_b    = hits_b / total_b
```

Exercise 7 — discussion

Smells present:

Exercise 7 — discussion

Smells present:

- 📋 **Duplicated code** — the same computation is repeated verbatim for each block.

Exercise 7 — discussion

Smells present:

- 📋 **Duplicated code** — the same computation is repeated verbatim for each block.

Exercise 7 — discussion

Smells present:

- 📋 **Duplicated code** — the same computation is repeated verbatim for each block.

Refactoring direction: Extract a reusable function; test it once. ***

Exercise 7 — one possible solution

```
def hit_rate(block):
    return sum(1 for t in block if t.correct) / len(block)

hr_a = hit_rate(block_a)
hr_b = hit_rate(block_b)
```

Exercise 8

```
def integrand_uniform(self, p):
    return self.likelihood(p) * self.uniform_prior(p)

def integrand_beta(self, p):
    return self.likelihood(p) * self.beta_prior(p)

def integrand_jeffreys(self, p):
    return self.likelihood(p) * self.jeffreys_prior(p)
```

Exercise 8 — discussion

Smells present:

Exercise 8 — discussion

Smells present:

- 🧩 **Shotgun surgery** — adding a prior requires adding another near-identical method.

Exercise 8 — discussion

Smells present:

- 🧩 **Shotgun surgery** — adding a prior requires adding another near-identical method.
- 📋 **Duplicated code** — the same `self.likelihood(p) * prior(p)` pattern repeats.

Exercise 8 — discussion

Smells present:

- 🧩 **Shotgun surgery** — adding a prior requires adding another near-identical method.
- 📋 **Duplicated code** — the same `self.likelihood(p) * prior(p)` pattern repeats.

Exercise 8 — discussion

Smells present:

- 🧩 **Shotgun surgery** — adding a prior requires adding another near-identical method.
- 📋 **Duplicated code** — the same `self.likelihood(p) * prior(p)` pattern repeats.

Refactoring direction: Extract the shared structure; pass the prior as an argument. ***

Exercise 8 — one possible solution

```
def integrand(self, prior, p):
    return self.likelihood(p) * prior(p)

# callers:
# self.integrand(self.uniform_prior,   p)
# self.integrand(self.beta_prior,      p)
# self.integrand(self.jeffreys_prior,  p)
```

Exercise 9

```
def compute_median(trial_list):
    trial_list.sort()
    n = len(trial_list)
    return trial_list[n // 2]

rts = [0.42, 0.31, 0.58, 0.29]
m   = compute_median(rts)
print(rts)  # [0.29, 0.31, 0.42, 0.58] — caller's list was mutated
```

Exercise 9 — discussion

Smells present:

Exercise 9 — discussion

Smells present:

- ⚡ Uncontrolled side effects — `trial_list.sort()` mutates the caller's list in place.

Exercise 9 — discussion

Smells present:

- ⚡ Uncontrolled side effects — `trial_list.sort()` mutates the caller's list in place.

Exercise 9 — discussion

Smells present:

- ⚡ Uncontrolled side effects — `trial_list.sort()` mutates the caller's list in place.

Refactoring direction: Work on a local copy; leave the caller's data unchanged. ***

Exercise 9 — one possible solution

```
def compute_median(trial_list):
    sorted_trials = sorted(trial_list)   # returns a new list
    n = len(sorted_trials)
    return sorted_trials[n // 2]

rts = [0.42, 0.31, 0.58, 0.29]
m   = compute_median(rts)
print(rts)  # [0.42, 0.31, 0.58, 0.29] — unchanged
```

Exercise 10

```
class SignalDetection:

    # Set the d-prime value
    def set_dprime(self, value):
        self.dprime = value  # store d-prime

    # Get the d-prime value
    def get_dprime(self):
        return self.dprime  # return the threshold
```

Exercise 10 — discussion

Smells present:

Exercise 10 — discussion

Smells present:

- 💬 **Excessive comments** — every method and line is annotated with what the code obviously does.

Exercise 10 — discussion

Smells present:

- 💬 Excessive comments — every method and line is annotated with what the code obviously does.
- 💬 “Code deodorant” — comments are masking a design issue rather than explaining intent.

Exercise 10 — discussion

Smells present:

- 💬 **Excessive comments** — every method and line is annotated with what the code obviously does.
- 💬 “**Code deodorant**” — comments are masking a design issue rather than explaining intent.
- 🏛️ **Lazy class** hint — bare getters/setters with no logic may not need to exist at all.

Exercise 10 — discussion

Smells present:

- 💬 **Excessive comments** — every method and line is annotated with what the code obviously does.
- 💬 “**Code deodorant**” — comments are masking a design issue rather than explaining intent.
- 🏛️ **Lazy class** hint — bare getters/setters with no logic may not need to exist at all.

Exercise 10 — discussion

Smells present:

- 💬 Excessive comments — every method and line is annotated with what the code obviously does.
- 💬 “Code deodorant” — comments are masking a design issue rather than explaining intent.
- 🏛️ Lazy class hint — bare getters/setters with no logic may not need to exist at all.

Refactoring direction: Remove comments that repeat the code; consider whether the accessors add value. ***

Exercise 10 — one possible solution

```
class SignalDetection:

    def __init__(self, dprime):
        self._dprime = dprime

    def set_dprime(self, value):
        # We don't want users to edit dprime per policy 22031.
        raise AttributeError("The dprime value cannot be edited.")

    def get_dprime(self):
        return self._dprime
```

Exercise 11

```
summary = {
    s: sum(t.rt for t in trials if t.subject==s)
       / len([t for t in trials if t.subject==s])
    for s in subjects
}
```

Exercise 11 — discussion

Smells present:

Exercise 11 — discussion

Smells present:

- 📜 **Excessively long line** — the expression is hard to read, debug, or reuse.

Exercise 11 — discussion

Smells present:

- 📜 **Excessively long line** — the expression is hard to read, debug, or reuse.
- 📋 **Duplicated subexpression** — the same filter `[t for t in trials if t.subject==s]` is written twice.

Exercise 11 — discussion

Smells present:

- 📜 **Excessively long line** — the expression is hard to read, debug, or reuse.
- 📋 **Duplicated subexpression** — the same filter `[t for t in trials if t.subject==s]` is written twice.

Exercise 11 — discussion

Smells present:

- 📜 Excessively long line — the expression is hard to read, debug, or reuse.
- 📋 Duplicated subexpression — the same filter `[t for t in trials if t.subject==s]` is written twice.

Refactoring direction: Extract a helper function; name the intermediate result. ***

Exercise 11 — one possible solution

```
def mean_rt(trials, subject):
    subject_trials = [t for t in trials if t.subject == subject]
    return sum(t.rt for t in subject_trials) / len(subject_trials)

summary = {s: mean_rt(trials, s) for s in subjects}
```

Exercise 12

```
from scipy.stats import norm

def analyse(sid, hit, far, mrt, srt, age, grp):
    if not (0 <= hit <= 1):
        raise ValueError("hit out of range")
    dp = norm.ppf(hit) - norm.ppf(far)
    return sid, dp, mrt / srt
```

Exercise 12 — discussion

Smells present:

Exercise 12 — discussion

Smells present:

- 📦 **Primitive obsession** — seven loose primitives that describe one participant; callers must remember the order.

Exercise 12 — discussion

Smells present:

- 📦 **Primitive obsession** — seven loose primitives that describe one participant; callers must remember the order.
- 🔡 **Short identifiers** — `sid`, `hit`, `far`, `mrt`, `srt`, `grp` are abbreviated without gain.

Exercise 12 — discussion

Smells present:

- 📦 **Primitive obsession** — seven loose primitives that describe one participant; callers must remember the order.
- 🔡 **Short identifiers** — `sid`, `hit`, `far`, `mrt`, `srt`, `grp` are abbreviated without gain.

Exercise 12 — discussion

Smells present:

- 📦 Primitive obsession — seven loose primitives that describe one participant; callers must remember the order.
- 🔡 Short identifiers — `sid`, `hit`, `far`, `mrt`, `srt`, `grp` are abbreviated without gain.

Refactoring direction: Group participant data into a dataclass; pass the object. ***

Exercise 12 — one possible solution (part 1)

```python
from dataclasses import dataclass

@dataclass
class Participant:
    subject_id: str
    hit_rate: float
    false_alarm_rate: float
    mean_rt: float
    sd_rt: float
    age: int
    group: str
```

Exercise 12 — one possible solution (part 2)

```
from scipy.stats import norm

def analyse(p: Participant):
    if not (0 <= p.hit_rate <= 1):
        raise ValueError("hit_rate out of range")
    dp = norm.ppf(p.hit_rate) - norm.ppf(p.false_alarm_rate)
    return p.subject_id, dp, p.mean_rt / p.sd_rt
```